Asadito RG: guía de desarrollo y práctica

Actualizada el 10/11/2026 para la versión con Java simplificado. La explicación completa de cada archivo Java está en CAMBIOS_JAVA_Y_SESIONES.md y CAMBIOS_JAVA_Y_SESIONES.pdf. Esa guía toma como referencia ModificaClient.pdf, compartido por el profesor.

1. Idea y funcionamiento

El proyecto compara precios de cortes de asado entre comercios de Río Grande. La persona selecciona cantidades de 0,5 kg en 0,5 kg y compara cuánto cuesta su lista completa en cada local. Un comercio sin precio de un corte seleccionado no participa del ranking: un precio faltante no equivale a cero.

En móviles se elige un corte y aparecen tarjetas ordenadas por precio por kilo, de menor a mayor. En escritorio se conserva la tabla. Los filtros y las confirmaciones flotantes pertenecen al frontend.

Se puede descargar un presupuesto PDF con el logo y el detalle por comercio. WhatsApp recibe el resumen mediante wa.me; el PDF se adjunta manualmente o mediante Web Share cuando el dispositivo lo admite. No se envían mensajes automáticamente.

2. Referencia a los criterios de la materia

Esta sección toma como orientación el PDF de criterios que compartiste. No garantiza aprobación ni sustituye requisitos específicos que indique la cátedra.

Criterio	Implementación para mostrar	Práctica sugerida
Sesiones	Spring Security, formulario /api/login, cookie JSESSIONID y /api/logout	Explicar el recorrido de email y contraseña hasta UserDetailsService.
Contraseñas	PasswordEncoder del ejemplo docente, hashes BCrypt	Diferenciar hash de cifrado y mostrar que no se devuelve el password.
ABM	ADMIN mantiene cortes/locales; OPERADOR publica precios	Crear, editar y dar de baja un registro.
Bajas lógicas	Campo activo y filtros de consulta	Verificar que la fila permanece en la base con activo=false.
Organización	Controladores, DTO, repositorios, modelos y servicio originales	Seguir un GET desde la pantalla hasta el DTO.
Entidades y POO	Locales y Productos relacionados mediante Precios; Usuario para acceso	Explicar relaciones, identidad, constructor y getter.
Validaciones	Condiciones breves en los controladores	Probar campos vacíos, precio negativo y fecha futura.
Frontend/backend	Fetch, formularios URL-encoded, sesión y CSRF	Revisar una solicitud en Network y su respuesta HTTP.
Compila y funciona	Build, pruebas y ejecución local	Probar el proyecto en tu propia computadora antes del examen.

Criterio	Implementación para mostrar	Práctica sugerida
Propuesta propia	Comparador, cantidades y presupuesto por comercio	Comparar la misma compra en dos locales.
Dominio del código	Guía de archivos y ejercicios	Modificar un límite sencillo y explicar dónde se aplica.

3. Qué queda en Java

El Java original se toma del commit 3ba7c00. Se restauran LocalesDTO, ProductosDTO, PreciosDTO y ProductoPrecioDTO; solo se agrega la asignación faltante del importe en el constructor de PreciosDTO. Los repositorios de locales/productos y PreciosService quedan idénticos al original.

Los controladores originales reciben @RequestParam y guardan a través de sus repositorios. Se agregan los métodos mínimos para los paneles y sus validaciones. No hay DTO adicionales para solicitudes ni servicios nuevos de catálogo, operadores, cálculo o PDF.

Usuario, UsuarioRepository, WebAuthentication, WebAuthorization, UsuarioController y ApiErrors son los seis archivos adicionales respecto de la versión original. El documento CAMBIOS_JAVA_Y_SESIONES explica cada uno y todos los cambios de los archivos originales.

El precio conserva float por el pedido de mantener el modelo original. La base conserva numeric(14,2), pero el tipo Java puede perder centavos a importes grandes. Un cambio futuro a BigDecimal debe estudiarse por separado.

4. Qué queda en JavaScript

presupuesto.js adapta los DTO originales, elige el precio más reciente por fecha e ID y calcula los presupuestos en centavos. reporte.js genera el PDF con jsPDF, fuentes y logo locales. app.js presenta el comparador, las tarjetas móviles y el resumen para compartir.

El botón de PDF consulta la sesión para solicitar ingreso en la interfaz. El archivo es un presupuesto orientativo generado en el navegador; no se trata de un comprobante certificado por el servidor. No existen endpoints /api/presupuestos/*.

session.js comparte la lógica de sesión y CSRF entre las páginas. gestion.js administra los formularios y apartados. Los permisos y la validación de lo que se guarda siguen en Java: manipular un botón no evita las reglas de Spring Security.

5. Ejecutarlo y preparar las cuentas

Seguí el README para configurar PostgreSQL y las variables de conexión. En IntelliJ elegí la configuración que ejecutás, abrí Environment variables y cargá valores sin comillas.

Para crear el primer ADMIN se usan ASADITO_ADMIN_EMAIL y ASADITO_ADMIN_PASSWORD. La contraseña debe tener al menos 10 caracteres y hasta 72 bytes UTF-8. Después de la creación, retiralas y reiniciá. Cambiarlas no restablece la contraseña de una cuenta que ya existe.

En /gestion.html ingresá como ADMIN y creá operadores desde su apartado. Opcionalmente se conservan ASADITO_OPERADOR_EMAIL y ASADITO_OPERADOR_PASSWORD para una primera cuenta con otro email. El registro público crea USER.

Si tu base ya funcionaba con los paneles anteriores, no se necesita borrar tablas ni repetir la migración. Se conservan la tabla usuarios, password_hash, los roles y hashes existentes. Si tu base es anterior al agregado de activo, revisá docs/migracion-v2.sql con un respaldo y la aplicación detenida. El script también redondea la columna de precios a dos decimales: revisá su efecto antes de usarlo.

Los 8 cortes y 13 locales iniciales vienen del proyecto original. No se inventan precios ni se importan los Excel automáticamente. Un catálogo sin precios muestra un estado vacío hasta que el operador los publique.

6. Pruebas manuales para el examen

1. Abrí la página sin sesión: debe mostrar el comparador y permitir calcular.

2. Registrá un USER, cerrá sesión y volvé a ingresar. No debe aparecer gestión.

3. Como ADMIN, creá y editá un corte y un local. Revisá sus respuestas y DTO en Network.

4. Creá un operador. El email repetido debe devolver 409 y un mensaje flotante.

5. Como OPERADOR, publicá dos precios para el mismo corte/local. El historial debe conservar ambos y el comparador elegir el más reciente.

6. Probá una fecha futura, un precio negativo y un corte inexistente. Java debe rechazarlos.

7. Como ADMIN intentá publicar un precio: la API debe devolver 403. Como OPERADOR intentá crear un local: también debe devolver 403.

8. Elegí 1,5 kg, calculá y descargá el PDF con sesión. Revisá importe, fecha, logo y comercio.

9. Abrí el sitio a 390 px: seleccioná un corte y comprobá las tarjetas ordenadas por precio.

10. Cerrá sesión y comprobá /api/auth/me. Una escritura sin CSRF debe rechazarse.

7. Verificación automatizada

```
bash gradlew build
# Con Node.js 22 o superior, para las funciones de cálculo:
node --test src/test/frontend/presupuesto.test.mjs
```

Gradle ejecuta 11 pruebas de Java en H2 aislada. Comprueban arranque, DTO, sesiones, compatibilidad de hashes, permisos, CSRF, validaciones y gestión. Las tres pruebas de JavaScript verifican selección de precios por fecha/ID, separación de locales, redondeo, cantidades y precios faltantes.

El flujo completo también se verificó con PostgreSQL y Chromium en una base temporal, incluida la creación de un operador y la descarga del PDF generado en el navegador. Esas comprobaciones no cargaron datos ficticios en la base de desarrollo.

8. Continuar por tu cuenta

Primero practicá cambios pequeños: texto de una validación, máximo del nombre o una columna del panel. Localizá el formulario, el controlador y el DTO que intervienen antes de cambiar algo.

Para el código de sesiones, explicá la diferencia entre Usuario de tu proyecto y User de Spring: uno representa la fila de tu base y el otro los datos que necesita el framework para autenticar. Spring compara el hash y mantiene la sesión.

Para errores de precios consultá DIAGNOSTICO_PRECIOS.md. Se retiró el sistema anterior de códigos DB_* para reducir el manejador Java; la causa concreta se investiga en la consola y en las restricciones de

PostgreSQL.

Como mejoras futuras quedan recuperación de contraseñas, límites de intentos, actualización de Spring Boot, paginación del historial y envío por WhatsApp Business. Este último requiere una integración y credenciales propias; abrir wa.me no equivale a enviar un archivo por una API.